MAINS AUCTION

CST2550

Lene Nystuen – M01021531
Gaia Fiorillo – M01008665
Elysia Bonney – M01005210
Samara (Sellaka) Cunha Santos – M01035975
Khalid Sheikh-Abdi – M00983482

19.04.2026

Table of Contents

1. Introduction 1
   1.1 Project description 1
   1.2 The team 1
   1.3 Report layout 1
2. Design 2
   2.1 Justification of selected data structures 2
   2.2 Analysis of key algorithms 2
      2.2.1 Binary Search Tree: Insert 2
      2.2.2 Binary Search Tree: Search 3
      2.2.3 Binary Search Tree: Price range query 3
      2.2.4 Hash table: Insert 3
      2.2.5 Hash table: Resize 3
      2.2.6 Hash table: Search 4
      2.2.7 Linked list: Add bid 4
      2.2.8 Linked list: Get highest bid 4
   2.3 Summary of time complexities 4
   2.4 AI chat agent and proxy server implementation 4
3. Testing 5
   3.1 Testing approach 5
   3.2 Testing tables 5
      3.2.1 Table of test cases 5
      3.2.2 Manual testing table 6
4. Conclusion 7

References .....

1. Introduction

1.1 Project description

At the beginning of this project, our group got allocated the task of creating an auction platform. We were free to choose whatever format we wanted, and what kind of items we wanted to auction. After brainstorming and discussing, we decided to focus on security. Therefore, we have created a property liquidation auction web application: MainsAuction. The full stack web application demonstrates custom data structures and auction logic, and is built on .NET10 with Blazor Server. The aim of our application is to help organisations liquidate assets quickly and transparently.

Some of the functionalities of our platform is users can browse and bid on live property auctions with real time countdowns, properties go through admin approval process before being listed, users can filter by location, property type, price and bedrooms, a bid history is tracked and we've implemented an AI chat agent to help guide users through the web application.

All the items for our project, like source code, daily standup meetings and video demonstration, have been added to our project's GitHub repo. Here is the link: www.github.com/gaiafiorillo/PropertyAuction.

1.2 The team

Our team members are Gaia Fiorillo, Lene Nystuen, Elysia Bonney, Khalid Sheikh-Abdi and Samara (Sellaka) Cunha Santos. Our team leader is Sellaka, our secretary is Elysia, our two developers are Gaia and Khalid, and our tester is Lene.

1.3 Report layout

This report consists of 4 chapters, with chapter 1 being introduction with a project and team description and an explanation of the report layout. Chapter 2 is design, consisting of justification of selected data structures, analysis of key algorithms, summary of time complexities and an explanation of our AI chat agent and proxy server implementation. Chapter 3 explains our testing approach and provides tables representing test cases and manual system testing. Finally, we conclude the report in chapter 4.

2. Design

2.1 Justification of selected data structures

The platform we built requires efficient storage and retrieval of three core entities: auctions, users, and bids. Three custom data structures were selected and implemented without the use of any standard template library, each chosen based on the specific access patterns of the entity it stores.

A Binary Search Tree (BST) was selected to store auction data. Auctions are frequently searched by ID and require ordered traversal for price range filtering. The BST provides O(log n) average case time complexity for insertion, search, and deletion, making it well suited to a system where auctions are regularly added and queried (Cormen et al., 2022; GeeksforGeeks, 2024). A linear structure such as an array would require O(n) search time, which becomes inefficient as the number of auctions grows. The BST also supports in order traversal in O(n), enabling price range queries without any additional sorting overhead.

A Hash Table was selected to store user data, implemented with separate chaining for collision resolution. User authentication requires fast lookup by username and the hash table provides O(1) average case search and insertion (Knuth, 1998; GeeksforGeeks, 2024). A BST was considered but rejected for user storage as there is no requirement to traverse users in sorted order, making the O(1) hash table preferable over O(log n) BST lookup. The implementation auto resises when the load factor reaches 0.75, maintaining consistent performance as the number of registered users grows.

A custom Linked List was selected to store bid history for each auction. Bids are always appended and retrieved in sequence and the linked list supports O(1) insertion at the head and O(n) full traversal to retrieve history or find the highest bid (Sedgewick and Wayne, 2011). A dynamic array was considered but rejected due to resising overhead. Since bid counts are unpredictable and growth is append only, the linked list is the more appropriate choice

2.2 Analysis of key algorithms

2.2.1 Binary Search Tree: Insert

The BST insert operation locates the correct position for a new auction by comparing the auction ID against each node, traversing left if the new ID is smaller and right if it is larger, until a null position is found. This recursive process gives an average time complexity of O(log n) as each comparison eliminates approximately half the remaining tree. In the worst case, if auctions are inserted in sorted order, the tree degenerates into a linear chain giving O(n) complexity (Cormen et al., 2022).

Pseudocode:

FUNCTION Insert(node, auction):

IF node IS NULL THEN

RETURN new Node(auction)

END IF

IF auction.ID < node.auction.ID THEN

node.left <-- Insert(node.left, auction)

ELSE IF auction.ID > node.auction.ID THEN

node.right <-- Insert(node.right, auction)

END IF

RETURN node

END FUNCTION

2.2.2 Binary Search Tree: Search

Search operates identically to insert, comparing the target auction ID against each node and traversing left or right accordingly. The search terminates either when the target ID is found or a null node is reached, indicating the auction does not exist. Average time complexity is O(log n), with the same O(n) worst case for a degenerate tree.

2.2.3 Binary Search Tree: Price range query

The price range query traverses the entire tree in O(n), visiting every node and collecting auctions whose current highest bid falls within the specified minimum and maximum. A full traversal is necessary because price is not the BST ordering key. The tree is ordered by auction ID, so there is no way to eliminate subtrees based on price alone. This is an accepted trade off as the BST is optimised for O(log n) ID based lookup, with price range queries accepting O(n) as the cost.

2.2.4 Hash table: Insert

The hash table insert operation computes a hash of the username key using a polynomial rolling hash function, producing an index into the underlying array. If a collision occurs, the new entry is prepended to the linked list at that bucket. After insertion, if the ratio of stored entries to table size exceeds the 0.75 load factor threshold, a resize operation is triggered. The average time complexity is O(1) and the worst case is O(n) if all keys hash to the same bucket, though the load factor threshold prevents this in practice (Knuth, 1998).

2.2.5 Hash table: Resize

When the load factor threshold is exceeded, the table doubles in size and all existing entries are rehashed into the new array. This operation costs O(n) as every entry must be reinserted, but occurs infrequently enough that the amortised cost of insertion remains O(1) (Cormen et al., 2022).

2.2.6 Hash table: Search

Search computes the hash of the target username to find the corresponding bucket, then traverses the linked list at that bucket comparing keys until a match is found or the list is exhausted. Average time complexity is O(1) and the worst case is O(n).

2.2.7 Linked list: Add bid

New bids are inserted at the head of the linked list by creating a new node, pointing its next reference to the current head, and updating the head pointer to the new node. This operation is always O(1) as it requires no traversal.

2.2.8 Linked list: Get highest bid

Finding the highest bid requires a full traversal of the linked list, maintaining a reference to the highest amount seen so far and updating it whenever a larger bid is encountered. Time complexity is O(n) where n is the number of bids placed on that auction. This is acceptable as bid counts per auction remain small relative to the total number of auctions in the system.

2.3 Summary of time complexities

Operation	**Data structure**	**average Case**	**worst case**
Insert auction	BST	O(log n)	O(n)
Search auction by ID	BST	O(log n)	O(n)
Delete auction	BST	O(log n)	O(n)
Price range query	BST	O(n)	O(n)
Insert user	Hash Table	O(1)	O(n)
Search user	Hash Table	O(1)	O(n)
Add bid	Linked List	O(1)	O(1)
Get highest bid	Linked List	O(n)	O(n)
Get all bids	Linked List	O(n)	O(n)

2.4 AI chat agent and proxy server implementation

The application includes an AI chat agent (ChatAgent.razor) powered by OpenAI's GPT-4o-mini model, which provides users with an interactive assistant capable of answering questions about the platform and the auction process. To avoid exposing the OpenAI API key within the source code or requiring markers to configure their own key to run the application, a lightweight proxy server was developed and deployed separately on Railway. The proxy server acts as a middleman between the Blazor application and the OpenAI API. When an user sends a message, the client makes an HTTP request to the proxy rather than directly to OpenAI. The proxy then forwards the request to the OpenAI API using the stored key on the

server side, receives the response, and returns it to the client. This means the API key is never exposed in the repository or the compiled application. The proxy URL is stored in appsettings.json under the key OpenAI:ProxyUrl, making it straightforward to reconfigure if needed.

3. Testing

3.1 Testing approach

Unit testing was the primary testing method applied, using the NUnit framework with the [TestFixture], [SetUp], and [Test] attributes. Each test class targets a single component, and the [SetUp] method creates a fresh instance before every test to ensure complete isolation; no test depends on the state left by another. Rather than using a mocking framework such as Moq, a FakeUserRepository was written to implement the IUserRepository interface, providing a lightweight in-memory substitute that keeps tests readable and avoids third-party dependencies. Tests were designed to prioritise meaningful coverage over quantity. Boundary conditions were specifically targeted: for example, the exact lockout threshold of five failed login attempts, the 15-minute unlock window, the three BST deletion cases (leaf, one child, two children), and the hash table resize at 75% load factor. A total of 91 unit tests were created. No formal integration testing was carried out as it fell outside the project requirements. Due to the limitations of this report's length, test cases will only be briefly represented in the table of test cases. Manual system testing was executed to verify end-to-end application behaviour, documented in chapter 3.2.2.

3.2 Testing tables

3.2.1 Table of test cases

ID	Scenario	Expected Result
AuctionBST		
TC1–2	Empty tree; single insert	IsEmpty()=true, Size=0; Size=1 after insert
TC3–5	Duplicate insert; search existing/missing	Duplicate ignored; correct node or null returned
TC6–7	Delete leaf/1-child/2-child; delete missing	Tree valid, size decreases; returns false
TC8–10	GetAllSorted; PriceRange search; Clear	Ascending order; inclusive range; resets to empty
UserHashTable		
TC11–13	Empty table; null insert; duplicate insert	IsEmpty; ArgumentException; existing user updated
TC14–16	Search existing/missing; delete existing/missing	Case-sensitive lookup; size decreases; false if absent
TC17–18	15 users (>0.75 load); Clear	All findable after resize; resets to empty
User Model		
TC19–20	Default state; SetPassword/VerifyPassword	Role=Bidder, IsLocked=false; hash stored, correct=true
TC21–23	Failed logins; reset; auto-unlock	Locks at 5 attempts; resets on success; unlocks after 15 min
BidList & BidService		
TC24–25	Add bids; GetAll/GetHighest	Most recent first; highest amount returned; null if empty
TC26–27	PlaceBid failures; PlaceBid success	False + message for invalid cases; true, bid stored
TC28–29	BidHistory per auction; GetUserBids	Histories independent; returns only specified user's bids
AuctionService		
TC30–31	GetAll; AddAuction/GetById	Min 4 auctions sorted; correct auction or null
TC32–33	GetActive; IsActive()	Only Active status; true only when Active & within time range

UserService		
TC34–35	RegisterAsync valid/invalid/duplicate; roles	Success or correct exception; role & hash stored
TC36–38	LoginAsync correct/wrong; lockout; reset	Returns user or null; locks at 5 fails; resets on success
TC39	LoadSampleUsersAsync once/twice	3 users created; InvalidOperationException on 2nd call

3.2.2 Manual testing table

ID	Scenario	Steps	Expected Result	Pass / Fail
ST01	User registration	Launch application. Register a new user with a valid username, email and password.	Account created successfully. User can log in with new credentials.	Pass *15.04.2026*
ST02	Duplicate registration rejected	Attempt to register with a username or email already in the system.	Error message displayed. No duplicate account created.	Pass *15.04.2026*
ST03	Login and lockout	Enter wrong password five times for a valid account. Then attempt login with correct password.	Account locked after fifth failure. Correct password rejected with locked message.	Pass *15.04.2026*
ST04	Place a bid on active auction	Log in as a bidder. Navigate to an active auction. Submit a bid higher than the current highest.	Bid accepted. Current highest bid updates to new amount.	Pass 15.*04.2026*
ST05	Bid rejected below current price	Log in as a bidder. Submit a bid equal to or lower than the current highest bid.	Bid rejected with appropriate message. Current highest bid unchanged.	Pass 15.*04.2026*
ST06	New bid added to bid history	Submit bid higher than current bid. Click property listing. View bid history.	New bid accepted and added to the list of bid history on property listing page.	Pass 15.*04.2026*
ST07	Search auctions by price range	Use the price range search with max value.	Only auctions with current highest bid within the specified range returned.	Pass 15.*04.2026*
ST08	Filter auctions by location, property type and min bedrooms	Use filter to search for specific type of properties.	Only auctions matching the filters criteria show up.	Pass 15.*04.2026*

4. Conclusion

To conclude, MainsAuction achieved its goal of delivering a secure and functional property liquidation auction platform, with custom data structures forming the backbone of the system. The Binary Search Tree, Hash Table, and Linked List each performed as intended, and the combination of 91 unit tests and manual system testing ensured that the key logic and edge cases were handled correctly. The platform supports user registration and authentication, active auction browsing, real-time bidding, and price range filtering, with an AI chat agent integrated via a secure proxy server.

Reflecting honestly on the process, there are several things we would approach differently. The most significant was database integration. A relational database was planned early on, however schema changes late in the development cycle caused integration difficulties that ultimately left the application relying entirely on in-memory data structures. As a result, data does not persist between sessions, which is a notable limitation. In future work, we would commit to a fixed schema earlier and treat database integration as a foundational step rather than something to be completed later in the project.

On the security side, while we implemented features such as account lockout and password hashing, there is clear room to go further. JWT-based authentication with session tokens would replace the current credential checking with a far more robust and scalable approach. Alongside this, comprehensive server-side input validation across all forms would be essential to guard against threats such as SQL injection, which becomes especially important once a real database is introduced.

From a testing perspective, although the unit testing was thorough, no formal integration testing was carried out as it fell outside the project requirements. In hindsight, even a limited set of integration tests would have helped surface the issues caused by the late schema changes before they became a problem.

Finally, clearer task allocation from the start would have made for a smoother development process overall, reducing the risk of work becoming unevenly distributed at critical points in the timeline. The lessons drawn from this experience, like committing to a database schema early, building security features in from the start, and structuring team responsibilities more deliberately, will help us form a better foundation for our future projects.

References

Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C. (2022) Introduction to Algorithms. 4th edn. Cambridge, MA: MIT Press.

DocFX (2018). *NUnit Documentation | NUnit Docs*. [online] Nunit.org. Available at: https://docs.nunit.org/articles/nunit/intro.html [Accessed 26 Feb. 2026].

Eissa, A. (2025). *CST 2550 - 2025: Software Engineering and Management - Week 7: Debugging and Testing Software*. [online] London: Middlesex University. Available at: https://mdx.mrooms.net/mod/resource/view.php?id=3290783 [Accessed Feb. 2026].

GeeksforGeeks (2024) Binary Search Tree Data Structure. Available at: https://www.geeksforgeeks.org/binary-search-tree-data-structure/ (Accessed: 20 February 2026).

GeeksforGeeks (2024) Hashing Data Structure. Available at: https://www.geeksforgeeks.org/hashing-data-structure/ (Accessed: 20 February 2026).

Knuth, D.E. (1998) The Art of Computer Programming, Volume 3: Sorting and Searching. 2nd edn. Reading, MA: Addison-Wesley.

Microsoft (2025). *Create, run, and customize C# unit tests - Visual Studio (Windows)*. [online] Microsoft.com. Available at: https://learn.microsoft.com/en-us/visualstudio/test/walkthrough-creating-and-running-unit-tests-for-managed-code?view=visualstudio [Accessed 26 Feb. 2026].

Microsoft (2025b). *Tutorial: Debug C# code and inspect data - Visual Studio (Windows)*. [online] Microsoft.com. Available at: https://learn.microsoft.com/en-us/visualstudio/get-started/csharp/tutorial-debugger?view=visualstudio&toc=%2Fvisualstudio%2Fdebugger%2Ftoc.json [Accessed 19 Mar. 2026].

Schmitt, J. (2021). *Unit testing vs integration testing*. [online] CircleCI. Available at: https://circleci.com/blog/unit-testing-vs-integration-testing/ [Accessed 25 Feb. 2026].

Sedgewick, R. and Wayne, K. (2011) Algorithms. 4th edn. Upper Saddle River, NJ: Addison-Wesley.